**Course Title:**

BIT302 Software Engineering

## Course Contents

**Unit 7: Software Testing (5 Hrs.)**
Introduction to Software Testing,
Development Testing,
Test-Driven Development,
Types of Testing

# Course Contents

## Unit 7: Software Testing (5 Hrs.)

Introduction to Software Testing,

8.1 Development Testing,
- 8.1.1 Unit testing
- 8.1.2 Choosing unit test cases
- 8.1.3 Component testing
- 8.1.4 System testing

8.2 Test-Driven Development,

8.3 Release testing
- 8.3.1 Requirements-based testing
- 8.3.2 Scenario testing
- 8.3.3 Performance testing

8.4 User Testing

Types of Testing

# Course Contents

## Objectives:

The objective of this chapter is to introduce software testing and software testing processes.

you will:

- understand the stages of testing from testing during development to acceptance testing by system customers;
- have been introduced to techniques that help you choose test cases that are geared to discovering program defects;
- understand test-first development, where you design tests before writing code and run these tests automatically;
- know about three distinct types of testing - component testing, system testing, and release testing;
- understand the distinctions between development testing and user testing.

# Course Contents

## Program/Software testing:

- Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use. It is a dynamic validation technique.
- When you test software, you execute a program using artificial data.
- You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.
- Can reveal the presence of errors NOT their absence.
- Testing is part of a more general verification and validation process, which also includes static validation techniques.
- **Static Validation:** Code reviews, static code analysis tools, requirement analysis, design inspections, and other methods that do not involve actual execution. **Dynamic Validation:** Unit testing, integration testing, system testing, acceptance testing, and other methods that involve executing the code and observing its behavior.

# Course Contents

## Program/Software testing:

- Software is tested to uncover errors that were made unintentionally as it was designed and constructed. Test can be done to the entire program as a whole or run tests only on a small part of it.
- A strategy for software testing is developed by the project manager, software engineers, and testing specialists.
- Testing often accounts for more project effort than any other software engineering action. If it is conducted haphazardly, time is wasted, unnecessary effort is expended, and even worse, errors sneak through undetected.
- It would therefore seem reasonable to establish a systematic strategy for testing software.

## Course Contents

**Program testing goals –**

**Differences between <u>Validation testing</u> and <u>Defect testing</u>**

## Course Contents

**Program testing goals -** When you test software, you are trying to do two things:

1. To demonstrate to the developer and the customer that the *software meets its requirements*.
   - For custom software, this means that there should be at least one test for every requirement in the requirements document.
   - For generic software products, it means that there should be tests for all of the system features, plus combinations of these features, that will be incorporated in the product release.
2. To discover situations in which the behavior of the software is incorrect or undesirable.
   - Defect testing is concerned with rooting out undesirable system behavior such as system crashes, unwanted interactions with other systems, incorrect computations and data corruption.

## Course Contents

**Validation and defect testing:**

The first goal leads to validation testing

- You expect the system to perform correctly using a given set of test cases that reflect the system's expected use.

The second goal leads to defect testing

- The test cases are designed to expose defects. The test cases in defect testing can be deliberately obscure and need not reflect how the system is normally used.

## Course Contents

**Testing process goals:**

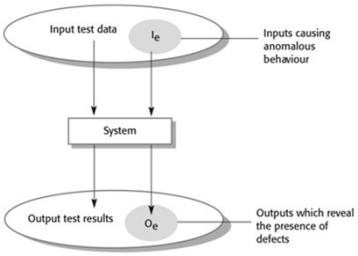
- Validation testing
  - To demonstrate to the developer and the system customer that the software meets its requirements
  - A successful test shows that the system operates as intended.
- Defect testing
  - To discover faults or defects in the software where its behaviour is incorrect or not in conformance with its specification
  - A successful test is a test that makes the system perform incorrectly and so exposes a defect in the system.

# Course Contents

## An input-output model of program testing:

Figure shows the differences between validation testing and defect testing. Think of the system being tested as a black box. The system accepts inputs from some input set I and generates outputs in an output set O. Some of the outputs will be erroneous. These are the outputs in set Oe that are generated by the system in response to inputs in the set Ie. The priority in defect testing is to find those inputs in the set Ie because these reveal problems with the system. Validation testing involves testing with correct inputs that are outside Ie. These stimulate the system to generate the expected correct outputs.

- Testing is part of a broader process of software verification and validation (V&V).
- Verification and validation are not the same thing, although they are often confused.




# Course Contents

## Verification and Validation (V&V):

- **Software verification** is the process of checking that the software meets its stated functional and non-functional requirements. Validation is a more general process. The aim of software validation is to ensure that the software meets the customer's expectations.
- Validation is essential because statements of requirements do not always reflect the real wishes or needs of system customers and users.

- **Verification:**

  "*Are we building the product right?*"

  - The software should conform to its specification.

- **Validation:**

  "*Are we building the right product?*"

  - The software should do what the user really requires.

# Course Contents

**V & V confidence:**

Aim of V & V is to establish confidence that the system is 'fit for purpose' and depends on:

Software purpose: The level of confidence depends on how critical the software is to an organization. The more critical the software, the more important it is that it is reliable. For example, the level of confidence required for software used to control a safety-critical system is much higher than that required for a demonstrator system that prototypes new product ideas.

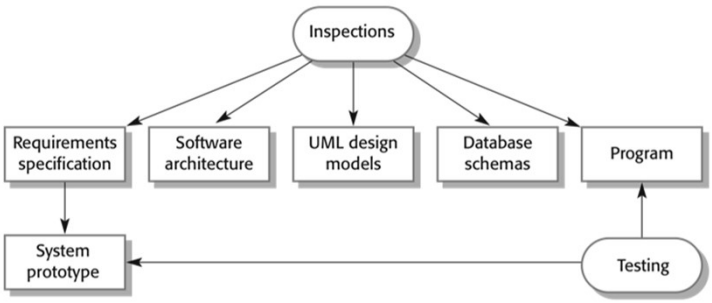
User expectations: Users may have low expectations of certain kinds of software.

Marketing environment: Getting a product to market early may be more important than finding defects in the program. In a competitive environment, the company may decide to release a program before it has been fully tested and debugged because it wants to be the first into the market. If a software product or app is very cheap, users may be willing to tolerate a lower level of reliability.

# Course Contents

**Inspections and Testing:**

- As well as software testing, the verification and validation process may involve software inspections and reviews. Inspections and reviews analyze and check the system requirements, design models, the program source code, and even proposed system tests. These are "static" V & V techniques in which you don't need to execute the software to verify it.



## Course Contents

**Inspections and Testing:**

- Software inspections: Concerned with analysis of the static system representation to discover problems (static verification)
  - May be supplemented by tool-based document and code analysis.
- Software testing: Concerned with exercising and observing product behaviour (dynamic verification)
  - The system is executed with test data and its operational behaviour is observed.

## Course Contents

**Software inspections:**

- These involve people examining the source representation with the aim of discovering anomalies and defects.
- Inspections do not require execution of a system so may be used before implementation.
- They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.)
- They have been shown to be an effective technique for discovering program errors.

# Course Contents

## Advantages of inspections:

- During testing, errors can mask (hide) other errors. Because inspection is a static process, you don't have to be concerned with interactions between errors.
- Incomplete versions of a system can be inspected without additional costs. If a program is incomplete, then you need to develop specialized test harnesses to test the parts that are available.
- As well as searching for program defects, an inspection can also consider broader quality attributes of a program, such as compliance with standards, portability and maintainability.

# Course Contents

## Inspections and Testing:

- Inspections and testing are complementary and not opposing verification techniques.
- Both should be used during the V & V process.
- Inspections can check conformance with a specification but not conformance with the customer's real requirements.
- Inspections cannot check non-functional characteristics such as performance, usability, etc.

## Course Contents

**Stages of testing:**

- Development testing, where the system is tested during development to discover bugs and defects.
- Release testing, where a separate testing team test a complete version of the system before it is released to users.
- User testing, where users or potential users of a system test the system in their own environment.

## What Is Verification and How Does It Work?

Verification is the process of checking that a software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not. It verifies whether the developed product fulfils the requirements that we have.

Verification is Static Testing. Activities involved in verification:

1. Inspections
2. Reviews
3. Walkthroughs
4. Desk-checking

Verification is the process of reviewing the intermediate work products of a software development lifecycle to ensure that we are on track to complete the final result. In other words, verification is a process of evaluating software mediation products to see whether they meet the requirements set out at the start of the phase.

## What Is Verification and How Does It Work?

- The issue now is, what exactly are **intermediate or mediator products**?
- These may comprise documentation like **requirements specifications**, **design documents**, **database table design**, **ER diagrams**, **test cases**, **traceability matrix**, and so on that are created throughout the development stages.
- We often overlook the necessity of examining these papers, but we should remember that reviewing may uncover numerous hidden flaws that, if discovered or addressed later in the development cycle, can be quite expensive.
- Verification uses review or non-executable techniques to guarantee that the system (software, hardware, documentation, and staff) meets an organization's standards and procedures.

## Where does verification take place?

Following are some of the areas in which verification is conducted in IT projects.

| Verification Situation | Actors | Definition | Output |
|---|---|---|---|
| **Requirement for Business/Functionality** | Examine the business needs with the development team and the customer. | This is a crucial stage to ensure that the requirements have been acquired and/or accurately, as well as to determine whether or not they are realistic. | The criteria have been finalized and are ready to be devoured by the following stage, design. |
| **Design Review** | Dev team | The Dev team extensively evaluates the design once it is created to ensure that the functional requirements can be satisfied with the design presented. | The design is ready for integration into an IT system. |
| **Code Walkthrough** | Individual Developer | After the code has been written, it is examined for any syntactic mistakes. This is a more informal task that is carried out by the individual developer on his or her own code. | Unit testing is now possible with this code. |

# Where does verification take place?

Following are some of the areas in which verification is conducted in IT projects.

| Verification Situation | Actors | Definition | Output |
|---|---|---|---|
| **Code Inspection** | Dev team | This is a more formal configuration. Subject matter experts and developers review the code to ensure that it meets the software's commercial and functional objectives. | The code is now available for testing. |
| **Test Plan Review (internal to QA team)** | QA Team | The QA team reviews a test plan internally to ensure that it is correct and thorough. | A test plan document that may be shared with other teams (Project Management, Business Analysis, development, Environment, client, etc.) |
| **Test Plan (External)** | Project Manager, Business Analyst, and Developer | A formal review of the test plan document to ensure that the QA team's timeframe and other factors are in sync with the other teams and the overall project. | A signed or authorized test plan document will be used to conduct the testing activity. |

# Where does verification take place?

Following are some of the areas in which verification is conducted in IT projects.

| Verification Situation | Actors | Definition | Output |
|---|---|---|---|
| **Test documentation review** | QA Team | A peer review is when members of a team check each other's work to ensure that the documentation is free of errors. | The test documentation is complete and ready to be shared with other teams. |
| **Test documentation final review** | Business Analyst and Development Team | A review of the test documentation to ensure that the test cases cover all of the system's business circumstances and functional features. | Test documentation is complete and ready to run. |

## What Is Validation and How Does It Work?

Validation is the process of assessing the completed product to see whether it fits the business requirements. To put it another way, the test execution that we undertake on a daily basis is essentially a validation activity that comprises smoke testing, functional testing, regression testing, systems testing, and so on.

Validation encompasses all types of testing that include interacting with the product and putting it to the test.

The validation procedures are listed below −

- Unit Testing
- Testing for integration
- System Evaluation
- Acceptance Testing of Users

Validation verifies that the system follows the plan by putting it through a set of tests that can be viewed and assessed.

## Examples of Validation and Verification

- Real-life Consider this scenario: You're going to a restaurant and ordering blueberry pancakes. How do you determine whether the meal that comes out is what you ordered when the waiter/waitress brings it out?

The first thing we notice when we look at it is the following −

  - Is the food what you'd expect pancakes to look like?
  - Are there any blueberries around?
  - Do they have the correct smell?

- Maybe a little more, but you get the idea.
- On the other hand, if you need to be certain that the dish is exactly what you anticipated, you'll have to consume it.
- When you haven't eaten yet but want to double-check a few things by going over the topics, verification is the way to go. When you validate a product, you actually consume it to verify whether it's correct.

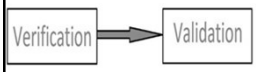
# Validation

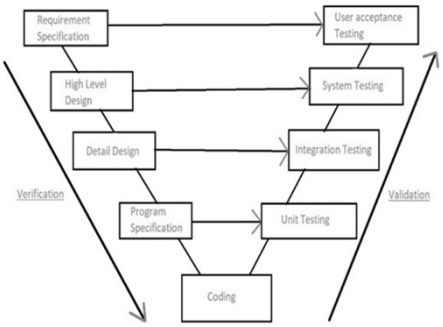
Validation is the process of checking whether the software product is up to the mark or in other words product has high level requirements. It is the process of checking the validation of product i.e., it checks what we are developing is the right product. it is validation of actual and expected product.

Validation is the **Dynamic Testing**. Activities involved in validation:

- Black box testing
- White box testing
- Unit testing
- Integration testing

Note: Verification is followed by Validation.




# Difference between Validation and Verification

| Verification | Validation |
|---|---|
| Examines intermediate products to see whether they fulfill the phase's specific criteria. | Examine the completed product to see whether it fulfills the company's requirements. |
| Checks if the product is constructed according to the requirements and design specifications. | It assesses if the software is suitable for usage and meets the requirements of the company. |
| "Are we creating the product correctly?" is checked. | "Are we developing the proper(Right) product?" is checked. |
| This is accomplished without running the program. | Is finished with the software's execution. |
| All static testing approaches are included. | All dynamic testing procedures are included. |
| Reviews, inspections, and walkthroughs are just a few examples. | All sorts of testing, such as smoke, regression, functional, systems, and UAT(User Acceptance Testing) are examples. |

## Techniques of Testing

Black-box and White-box Testing, Inspections

Black Box Testing:- Black Box Testing is a software testing method in which the functionalities of software applications are tested without having knowledge of internal code structure, implementation details and internal paths. Black Box Testing mainly focuses on input and output of software applications and it is entirely based on software requirements and specifications. It is also known as Behavioural Testing.

The above Black-Box can be any software system you want to test. For Example, an operating system like Windows, a website like Google, a database like Oracle or even your own custom application. Under Black Box Testing, you can test these applications by just focusing on the inputs and outputs without knowing their internal code implementation.

## Techniques of Testing

**How to do Black-box Testing?**

Here are the generic steps followed to carry out any type of Black Box Testing.

- Initially, the requirements and specifications of the system are examined.
- Tester chooses valid inputs (positive test scenario) to check whether Software under Test (SUT) processes them correctly. Also, some invalid inputs (negative test scenario) are chosen to verify that the SUT is able to detect them.
- Tester determines expected outputs for all those inputs.
- Software tester constructs test cases with the selected inputs.
- The test cases are executed.
- Software tester compares the actual outputs with the expected outputs.

Defects if any are fixed and re-tested.

## Techniques of Testing

### Types of Black Box Testing:

There are many types of Black Box Testing but the following are the prominent ones –

- **Functional testing –** This black box testing type is related to the functional requirements of a system; it is done by software testers.
- **Non-functional testing –** This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, usability.
- **Regression testing –** Regression Testing is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.

## Techniques of Testing

### Black Box Testing Techniques:

Following are the prominent Test Strategy amongst the many used in Black box Testing.

- Equivalence Class Testing
- Boundary Value Testing
- Decision Table Testing

## Techniques of Testing

**Black Box Testing Techniques:**

Equivalence Class Testing:

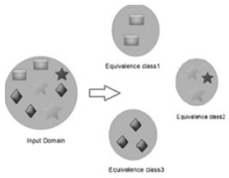
- Boundary Value Testing
- Decision Table Testing

## Techniques of Testing

**Black Box Testing Techniques:**

**Equivalence Class Testing:** It is used to minimize the number of possible test cases to an optimum level while maintains reasonable test coverage.

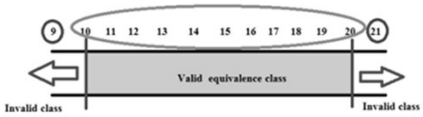
- It divides the input domain of a program into classes. Each class is called as equivalence partition because all the elements in an equivalence class test the same thing.
- If one element in an equivalence class catches a bug, others also probably do the same. Also, if one element doesn't catch a bug, others won't either.
- All the members of each equivalence class share some characteristics in common. These characteristics cannot be shared with the members of other classes.



# Techniques of Testing

## Black Box Testing Techniques:

**Boundary Value Testing:** Boundary value testing is focused on the values at boundaries. This technique determines whether a certain range of values are acceptable by the system or not. It is very useful in reducing the number of test cases. It is most suitable for the systems where an input is within certain ranges.




# Techniques of Testing

## Black Box Testing Techniques:

**Decision Table Testing:** A decision table puts causes and their effects in a matrix. There is a unique combination in each column.

A company's employees are paid bonuses if they work more than a year in the company and achieve individually agreed targets. Try to list out all the conditions and actions mentioned in the requirements. Hopefully, you would have listed out the conditions as listed below.

**Conditions:**

- Employment for more than 1 year?
- Agreed target?
- Achieved target?

**Actions / Outcome:**

- Bonus payment?
- With all these details, let's draw the decision table.

Decision Table

| | Rules | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Conditions** | **R1** | **R2** | **R3** | **R4** | **R5** | **R6** | **R7** | **R8** |
| C1:Employment for more than 1 year? | YES | NO | YES | NO | YES | NO | YES | NO |
| C2:Agreed target? | NO | NO | YES | YES | NO | NO | YES | YES |
| C3:Achieved target? | NO | NO | NO | NO | YES | YES | YES | YES |
| **Actions** | | | | | | | | |
| A1:Bonus payment? | NO | NO | NO | NO | NO | NO | YES | NO |

## Techniques of Testing

### How do you perform White Box Testing?

To give you a simplified explanation of white box testing, we have divided it into **two basic steps**. This is what testers do when testing an application using the white box testing technique:

**STEP 1) Understand the source code**

The first thing a tester will often do is learn and understand the source code of the application. Since white box testing involves the testing of the inner workings of an application, the tester must be very knowledgeable in the programming languages used in the applications they are testing. Also, the testing person must be highly aware of secure coding practices. Security is often one of the primary objectives of testing software. The tester should be able to find security issues and prevent attacks from hackers and naive users who might inject malicious code into the application either knowingly or unknowingly.

## Techniques of Testing

### How do you perform White Box Testing?

**Step 2) Create test cases and execute**

The second basic step to white box testing involves testing the application's source code for proper flow and structure. One way is by writing more code to test the application's source code. The tester will develop little tests for each process or series of processes in the application. This method requires that the tester must have intimate knowledge of the code and is often done by the developer. Other methods include Manual Testing, trial, and error testing and the use of testing tools as we will explain further on in this article.

# Techniques of Testing

## WhiteBox Testing Example:

Consider the following piece of code

```
------------  Printme is a function
Printme (int a, int b) {
   int result = a+ b;
   If (result> 0)
        Print ("Positive", result)
   Else
        Print ("Negative", result)
   }
-----------   End of the source code
```

The goal of WhiteBox testing in software engineering is to verify all the decision branches, loops, statements in the code.

To exercise the statements in the above white box testing example, WhiteBox test cases would be

A = 1, B = 1

A = -1, B = -3

# Techniques of Testing

**White Box Testing Techniques:**

Following are important WhiteBox Testing Techniques:

- Statement Coverage
- Decision Coverage
- Branch Coverage
- Condition Coverage
- Multiple Condition Coverage
- Finite State Machine Coverage
- Path Coverage
- Control flow testing
- Data flow testing

## Techniques of Testing

**Types of White Box Testing:**

*White box testing* encompasses several testing types used to evaluate the usability of an application, block of code or specific software package. There are listed below —

**Unit Testing:** It is often the first type of testing done on an application. Unit Testing is performed on each unit or block of code as it is developed. Unit Testing is essentially done by the programmer. As a software developer, you develop a few lines of code, a single function or an object and test it to make sure it works before continuing Unit Testing helps identify a majority of bugs, early in the software development lifecycle. Bugs identified in this stage are cheaper and easy to fix.
threats

## Techniques of Testing

**Types of White Box Testing:**

**Testing for Memory Leaks**: Memory leaks are leading causes of slower running applications. A QA specialist who is experienced at detecting memory leaks is essential in cases where you have a slow running software application.

**White Box Penetration Testing:** In this testing, the tester/developer has full information of the application's source code, detailed network information, IP addresses involved and all server information the application runs on. The aim is to attack the code from several angles to expose security threats

## Techniques of Testing

**Difference between Alpha and Beta Testing:**

- **Alpha Testing** is a type of software testing performed to identify bugs before releasing the product to real users or to the public. Alpha Testing is one of the user acceptance testing.
- **Beta Testing** is performed by real users of the software application in a real environment. Beta testing is one of the types of User Acceptance Testing.

## Techniques of Testing

**The difference between Alpha and Beta Testing is as follow:**

| Alpha Testing | Beta Testing |
|---|---|
| Alpha testing involves both the white box and black box testing. | Beta testing commonly uses black box testing. |
| Alpha testing is performed by testers who are usually internal employees of the organization. | Beta testing is performed by clients who are not part of the organization. |
| Alpha testing is performed at developer's site. | Beta testing is performed at end-user of the product. |
| Reliability and security testing are not checked in alpha testing. | Reliability, security and robustness are checked during beta testing. |
| Alpha testing ensures the quality of the product before forwarding to beta testing. | Beta testing also concentrates on the quality of the product but collects users input on the product and ensures that the product is ready for real time users. |
| Alpha testing requires a testing environment or a lab. | Beta testing doesn't require a testing environment or lab. |
| Alpha testing may require long execution cycle. | Beta testing requires only a few weeks of execution. |
| Developers can immediately address the critical issues or fixes in alpha testing. | Most of the issues or feedback collected from beta testing will be implemented in future versions of the product. |

## Techniques of Testing

**Regression Testing:**

**Regression Testing** is the process of testing the modified parts of the code and the parts that might get affected due to the modifications to ensure that no new errors have been introduced in the software after the modifications have been made. Regression means return of something and in the software field, it refers to the return of a bug.

**When to do regression testing?**

- When a new functionality is added to the system and the code has been modified to absorb and integrate that functionality with the existing code.
- When some defect has been identified in the software and the code is debugged to fix it.
- When the code is modified to optimize its working.

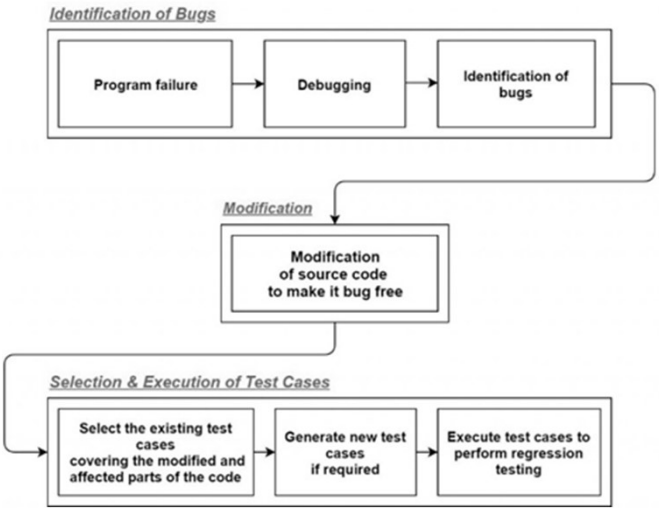
## Techniques of Testing

**Process of Regression testing:**

- Firstly, whenever we make some changes to the source code for any reasons like adding new functionality, optimization, etc. then our program when executed fails in the previously designed test suite for obvious reasons.
- After the failure, the source code is debugged in order to identify the bugs in the program. After identification of the bugs in the source code, appropriate modifications are made.
- Then appropriate test cases are selected from the already existing test suite which covers all the modified and affected parts of the source code.
- We can add new test cases if required. In the end regression testing is performed using the selected test cases.

## Techniques of Testing

**Process of Regression testing:**




## Techniques of Testing

**Design of Test Cases:**

A Test Case is a set of actions executed to verify a particular feature or functionality of software application. A Test Case contains test steps, test data, pre-condition, post-condition developed for specific test scenario to verify any requirement. The test case includes specific variables or conditions, using which a testing engineer can compare expected and actual results to determine whether a software product is functioning as per the requirements of the customer.

## Techniques of Testing

**Design of Test Cases:**

Test scenarios are rather vague and cover a wide range of possibilities. Testing is all about being very specific.

For a **Test Scenario**: To Check Login Functionality, many possible test cases are:

- **Test Case 1:** Check results on entering valid User Id & Password
- **Test Case 2:** Check results on entering Invalid User ID & Password
- **Test Case 3:** Check response when a User ID is Empty & Login Button is pressed, and many more

This is nothing but a Test Case.

## Techniques of Testing

**Create a Test Case for the scenario: Check Login Functionality:**

Step 1) A simple test case to explain the scenario would be

| Test Case # | Test Case Description |
|---|---|
| 1 | Check response when valid email and password is entered |

Step 2) Test the Data.

In order to execute the test case, you would need Test Data. Adding it below

| Test Case # | Test Case Description | Test Data |
|---|---|---|
| 1 | Check response when valid email and password is entered | Email: guru99@email.com<br>Password: lNf9^Oti7^2h |

## Techniques of Testing

**Create a Test Case for the scenario: Check Login Functionality:**

**Step 3)** Perform actions

In order to execute a test case, a tester needs to perform a specific set of actions on the AUT. This is documented as below:

| Test Case # | Test Case Description | Test Steps | Test Data |
|---|---|---|---|
| 1 | Check response when valid email and password is entered | 1) Enter Email Address<br>2) Enter Password<br>3) Click Sign in | Email: guru99@email.com<br>Password: lNf9^Oti7^2h |

## Course Contents

**Development testing:**

- Development testing includes all testing activities that are carried out by the team developing the system.
  - Unit testing, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
  - Component testing, where several individual units are integrated to create composite components. Component testing should focus on testing component interfaces.
  - System testing, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on testing component interactions.

## Course Contents

**1. Unit testing:**

- Unit testing is the process of testing individual components in isolation.
- It is a defect testing process.
- Units may be:
  - Individual functions or methods within an object
  - Object classes with several attributes and methods
  - Composite components with defined interfaces used to access their functionality.

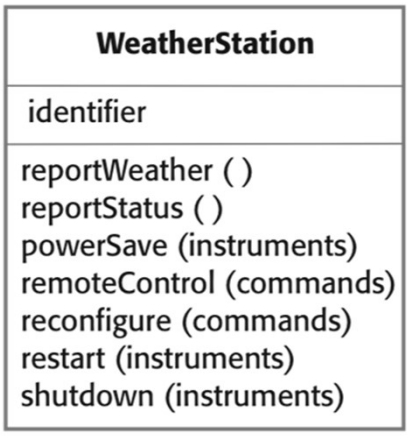
## Course Contents

**1. Unit testing: Object class testing**

- Complete test coverage of a class involves
  - Testing all operations associated with an object
  - Setting and interrogating all object attributes
  - Exercising the object in all possible states.
- Inheritance makes it more difficult to design object class tests as the information to be tested is not localized.

# Course Contents

**The weather station object interface:**




# Course Contents

**Weather station testing:**

- Need to define test cases for all operations.
- Using a state model, identify sequences of state transitions to be tested and the event sequences to cause these transitions
- For example:
  - Shutdown -> Running-> Shutdown
  - Configuring-> Running-> Testing -> Transmitting -> Running
  - Running-> Collecting-> Running-> Summarizing -> Transmitting -> Running

## Course Contents

**1. Unit testing: Unit test effectiveness**

- The test cases should show that, when used as expected, the component that you are testing does what it is supposed to do.
- If there are defects in the component, these should be revealed by test cases.
- This leads to 2 types of unit test cases:
  - The first of these should reflect normal operation of a program and should show that the component works as expected.
  - The other kind of test case should be based on testing experience of where common problems arise. It should use abnormal inputs to check that these are properly processed and do not crash the component.

## Course Contents

**Testing strategies:**

- Partition testing, where you identify groups of inputs that have common characteristics and should be processed in the same way.
  - You should choose tests from within each of these groups.
- Guideline-based testing, where you use testing guidelines to choose test cases.
  - These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.

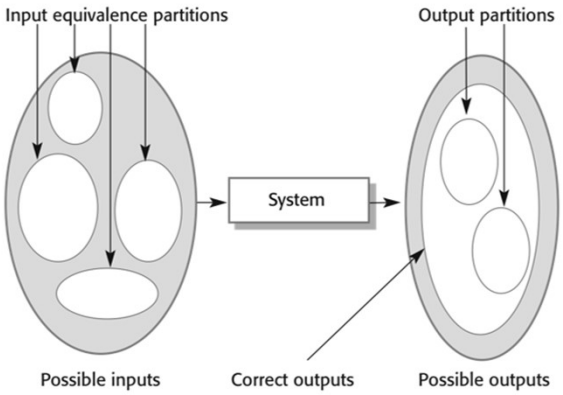
# Course Contents

**Partition testing:**

- Input data and output results often fall into different classes where all members of a class are related.
- Each of these classes is an equivalence partition or domain where the program behaves in an equivalent way for each class member.
- Test cases should be chosen from each partition.

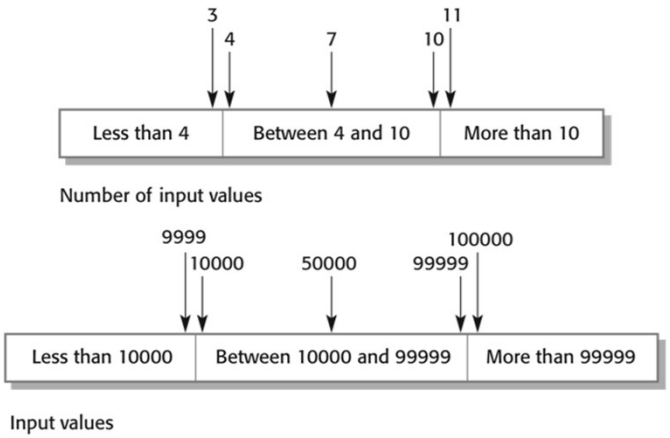
# Course Contents

**Equivalence partitioning:**



# Course Contents

**Equivalence partitions:**




# Course Contents

**Testing guidelines (sequences):**

- Test software with sequences which have only a single value.
- Use sequences of different sizes in different tests.
- Derive tests so that the first, middle and last elements of the sequence are accessed.
- Test with sequences of zero length.

## Course Contents

**General testing guidelines:**

- Choose inputs that force the system to generate all error messages
- Design inputs that cause input buffers to overflow
- Repeat the same input or series of inputs numerous times
- Force invalid outputs to be generated
- Force computation results to be too large or too small.

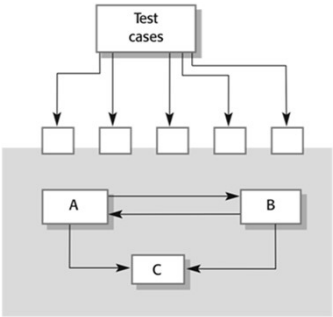
## Course Contents

**2. Component testing:**

- Software components are often composite components that are made up of several interacting objects.
  - For example, in the weather station system, the reconfiguration component includes objects that deal with each aspect of the reconfiguration.
- You access the functionality of these objects through the defined component interface.
- Testing composite components should therefore focus on showing that the component interface behaves according to its specification.
  - You can assume that unit tests on the individual objects within the component have been completed.

# Course Contents

**Interface testing:**




# Course Contents

**Interface testing:**

- Objectives are to detect faults due to interface errors or invalid assumptions about interfaces.
- Interface types
  - Parameter interfaces Data passed from one method or procedure to another.
  - Shared memory interfaces Block of memory is shared between procedures or functions.
  - Procedural interfaces Sub-system encapsulates a set of procedures to be called by other sub-systems.
  - Message passing interfaces Sub-systems request services from other sub-systems

## Course Contents

**Interface errors:**

- Interface misuse
  - A calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order.
- Interface misunderstanding
  - A calling component embeds assumptions about the behaviour of the called component which are incorrect.
- Timing errors
  - The called and the calling component operate at different speeds and out-of-date information is accessed.

## Course Contents

**Interface testing guidelines:**

- Design tests so that parameters to a called procedure are at the extreme ends of their ranges.
- Always test pointer parameters with null pointers.
- Design tests which cause the component to fail.
- Use stress testing in message passing systems.
- In shared memory systems, vary the order in which components are activated.

## Course Contents

**3. System testing:**

- System testing during development involves integrating components to create a version of the system and then testing the integrated system.
- The focus in system testing is testing the interactions between components.
- System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.
- System testing tests the emergent behaviour of a system.

## Course Contents

**System and component testing:**

- During system testing, reusable components that have been separately developed and off-the-shelf systems may be integrated with newly developed components. The complete system is then tested.
- Components developed by different team members or sub-teams may be integrated at this stage. System testing is a collective rather than an individual process.
  - In some companies, system testing may involve a separate testing team with no involvement from designers and programmers.

## Course Contents

**Use-case testing:**

- The use-cases developed to identify system interactions can be used as a basis for system testing.
- Each use case usually involves several system components so testing the use case forces these interactions to occur.
- The sequence diagrams associated with the use case documents the components and interactions that are being tested.

## Course Contents

**Testing policies:**

- Exhaustive system testing is impossible so testing policies which define the required system test coverage may be developed.
- Examples of testing policies:
  - All system functions that are accessed through menus should be tested.
  - Combinations of functions (e.g. text formatting) that are accessed through the same menu must be tested.
  - Where user input is provided, all functions must be tested with both correct and incorrect input.

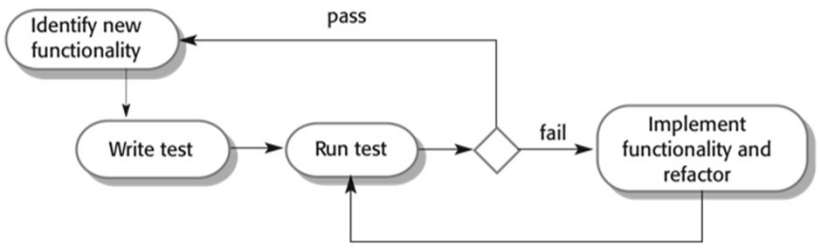
# Course Contents

**Test-driven development:**

- Test-driven development (TDD) is an approach to program development in which you inter-leave testing and code development.
- Tests are written before code and ‘passing’ the tests is the critical driver of development.
- You develop code incrementally, along with a test for that increment. You don’t move on to the next increment until the code that you have developed passes its test.
- TDD was introduced as part of agile methods such as Extreme Programming. However, it can also be used in plan-driven development processes.

# Course Contents

**Test-driven development:**



## Course Contents

**TDD process activities:**

- Start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
- Write a test for this functionality and implement this as an automated test.
- Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.

## Course Contents

**Benefits of test-driven development:**

- Code coverage
  - Every code segment that you write has at least one associated test so all code written has at least one test.
- Regression testing
  - A regression test suite is developed incrementally as a program is developed.
- Simplified debugging
  - When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.
- System documentation
  - The tests themselves are a form of documentation that describe what the code should be doing.

## Course Contents

**Regression testing:**

- Regression testing is testing the system to check that changes have not ‘broken’ previously working code.
- In a manual testing process, regression testing is expensive but, with automated testing, it is simple and straightforward. All tests are rerun every time a change is made to the program.
- Tests must run ‘successfully’ before the change is committed.

## Course Contents

**Release testing:**

- Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.
- The primary goal of the release testing process is to convince the supplier of the system that it is good enough for use.
  - Release testing, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.
- Release testing is usually a black-box testing process where tests are only derived from the system specification.

## Course Contents

**Release testing and system testing:**

- Forms of release testing:
  - Requirements-based testing
  - Scenario-based testing
- Release testing is a form of system testing.
- Important differences:
  - A separate team that has not been involved in the system development, should be responsible for release testing.
  - System testing by the development team should focus on discovering bugs in the system (*defect testing*). The objective of release testing is to check that the system meets its requirements and is good enough for external use (*validation testing*).

## Course Contents

**Requirements based testing:**

- Requirements-based testing involves examining each requirement and developing a test or tests for it.

## Course Contents

**Performance testing:**

- Part of release testing may involve testing the emergent properties of a system, such as performance and reliability.
- Tests should reflect the profile of use of the system.
- Performance tests usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable.
- Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behavior.

## Course Contents

**User testing:**

- User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
  - The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system. These cannot be replicated in a testing environment.

## Course Contents

**Types of user testing:**

- Alpha testing
  - Users of the software work with the development team to test the software at the developer's site.
- Beta testing
  - A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- Acceptance testing
  - Customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment. Primarily for custom systems.

## Course Contents

**Key points:**

- Testing can only show the presence of errors in a program. It cannot demonstrate that there are no remaining faults.
- Development testing is the responsibility of the software development team. A separate team should be responsible for testing a system before it is released to customers.
- Development testing includes unit testing, in which you test individual objects and methods, component testing in which you test related groups of objects, and system testing, in which you test partial or complete systems.

# Course Contents

**Key points:**

- When testing software, you should try to ‘break’ the software by using experience and guidelines to choose types of test case that have been effective in discovering defects in other systems.
- Wherever possible, you should write automated tests. The tests are embedded in a program that can be run every time a change is made to a system.
- Test-first development is an approach to development where tests are written before the code to be tested.
- Scenario testing involves inventing a typical usage scenario and using this to derive test cases.
- Acceptance testing is a user testing process where the aim is to decide if the software is good enough to be deployed and used in its operational environment.